6.5210 Final Project

Yifan Kang, Alex Zhao

November 2024

§1 Project Proposal

We will focus on the Single-Source Shortest Path (SSSP) problem in undirected graphs with real non-negative edge weights.

Previously, we believed that using Dijkstra's algorithm with Fibonacci heaps to achieve a $O(m + n \log n)$ runtime was optimal. However, in 2023 Duan et al. [1] discovered a randomized algorithm with a runtime of $O(m\sqrt{\log n \log\log n})$, which is faster than Fibonacci heap Dijkstra in the case where the average degree is smaller than $\sqrt{\log n}$.

The project would examine this new algorithm in detail and compare it with prior approaches, such as Fibonacci heap and binary heap Dijkstra. The focus will be synthesizing insights from the paper and related work, explaining the algorithm's key ideas, and making them more accessible to readers less familiar with these advanced algorithmic techniques.

§2 Randomized algorithm operation

§2.1 Algorithm Definition

The bottleneck of Dijkstra-based algorithms is the priority queue. Therefore, we break the original time bound by adding only a fraction of vertices into the priority queue. Let this small subset of vertices be R with $|R| = \frac{n}{k}$. Using this subset, we find the shortest path to other vertices by forming disjoint **bundles** based on R. Intuitively, the bundles should include vertices near its center so that we can calculate the shortest path effectively.

We introduced another concept of Ball(v) as the set of vertices closer to v than any vertices in R. We will later prove the fact that a vertex v will either have its shortest path through its bundle or through some vertex in its ball not too far from what we have previously calculated.

Definition 2.1. For all vertices $v \in V$, we find $b(v) = \text{argmin}_{u \in R} \text{dist}(v, u)$. $b(v)$ is its nearest neighbor in R.

Definition 2.2 (Ball)**.** $\text{Ball}(v) = \{\text{dist}(v, u) < \text{dist}(v, b(v)) | u \in V\}$ is the set of vertices closer than $b(v)$ to v (Figure 1).

Definition 2.3 (Bundle)**.** For $u \in R$, $\text{Bundle}(u) = \{b(v) = u | v \in V\}$ is the set of vertices v such that u is their nearest neighbor in R (Figure 2).

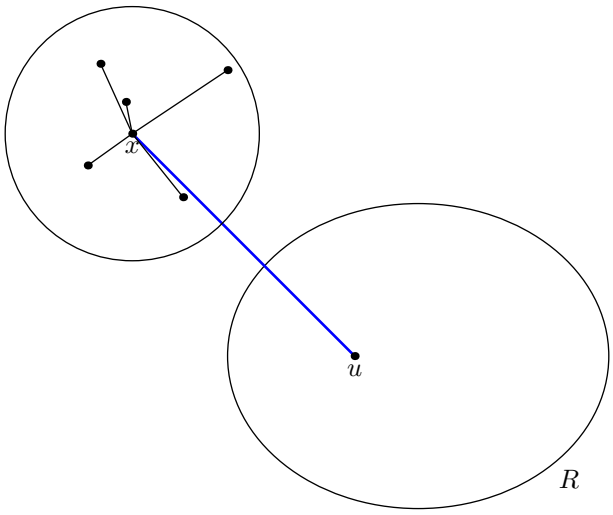

Figure 1: Figure demonstrating example for a ball.

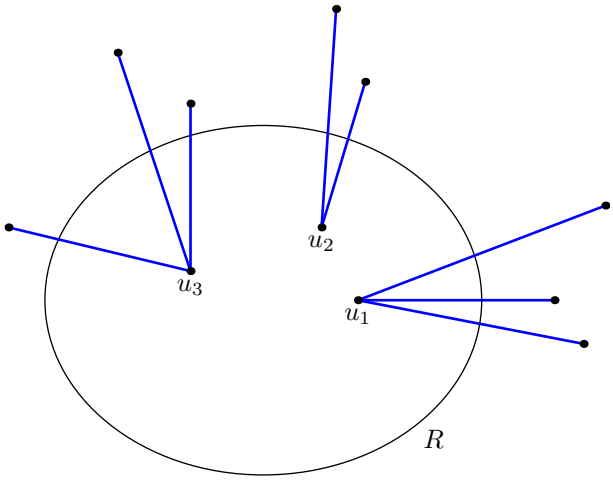

Figure 2: Figure demonstrating example for a bundle.

The algorithm can be divided into two parts: **bundle construction** and **bundle Dijkstra**.

Algorithm 2.4 (Bundle Construction) — Proceeds as follows:

1. We randomly sample a subset of vertices R.
2. For all vertices $v \in V$, we find $b(v)$ by running Dijkstra from v until we get to a point in R.
3. We also get $\text{dist}(v, w)$ for all $w \in \text{Ball}(v) \cup \{b(v)\}$ in the previous Dijkstra.

Definition 2.5 (Relax)**.** In our shortest path algorithm, relaxing a vertex v through u means updating $d(v)$ by $\min(d(v), d(u) + w_{u,v})$. This definition can be extended to relaxing a vertex v through the path $u_0, u_1, \ldots, u_k = v$, where we will update $d(v)$ by $\min(d(v), d(u_0) + \sum_{i=1}^{k} w_{u_{i-1},u_i})$. We use $\text{dist}(u, v)$ in place of $w_{u,v}$ if the value is known.

Algorithm 2.6 (Bundle Dijkstra) — Initially, we set $d(s) = 0$ and $d(v) = \infty$ for all $v \in V \setminus \{s\}$, and insert all $v \in R$ to a Fibonacci heap. When we pop a vertex u from the heap, we proceed as follows:

1.
   - For all $v \in \text{Bundle}(u)$, relax v by u.
   - For all above v, for all $y \in \text{Ball}(v)$, relax v by y.
   - For all $z_2 \in \text{Ball}(v) \cup \{v\}$ and $z_1 \in N(z_2)$, relax v by z_1, z_2.
2. For all $x \in \text{Bundle}(u)$, update $y \in N(x)$ and $z_1 \in \text{Ball}(y)$. We relax y by x and z_1 by x, y.
3. When we update $v \notin R$, we relax $b(v)$ by v.

§2.2 Algorithm Pseudocode

Algorithm 2.7 (Bundle Construction) — Setup for the algorithm.

Input: A graph $G = (V, E, w)$, source vertex $s \in V$, and parameter k
Output: Bundles and sets R, $Ball(v)$, $Bundle(u)$, $b(v)$
Initialize $R = \{s\}$ and insert each $v \in V \setminus \{s\}$ independently to R with probability $\frac{1}{k}$.
for all $v \in V \setminus R$ **do**
    Initialize Fibonacci heap $H^{(v)}$ with vertex v and its key $d^{(v)}(v) \leftarrow 0$
    Initialize an ordered list $V^{(v)}_{\text{extract}} \leftarrow \emptyset$
    while $H^{(v)}$ is not empty **do**
        $u \leftarrow H^{(v)}$.ExtractMin()
        Append u to $V^{(v)}_{\text{extract}}$
        if $u \in R$ **then**
            break the while-loop
        else
            for all $x \in N(u)$ **do**
                if $x \notin H^{(v)}$ and $x \notin V^{(v)}_{\text{extract}}$ **then**
                    $H^{(v)}$.Insert$(x, d^{(v)}(u) + w_{ux})$
                else if $d^{(v)}(x) > d^{(v)}(u) + w_{ux}$ **then**
                    $H^{(v)}$.DecreaseKey$(x, d^{(v)}(u) + w_{ux})$
for all $v \in V \setminus R$ **do**
    $b(v) \leftarrow$ the first vertex in the ordered list $V^{(v)}_{\text{extract}}$ that lies in R
Compute Bundle(u) for all $u \in R$, Ball(v) for all $v \in V \setminus R$, and record dist(v, u) for all $u \in \text{Ball}(v) \cup \{b(v)\}$

Algorithm 2.8 (Bundle Dijkstra) — Main algorithm.

Input: A graph $G = (V, E, w)$ and starting vertex $s \in V$
Output: Distance $d(v)$ from s to v for every vertex $v \in V$
Construct Bundles using Bundle Construction.
Set label $d(s) \leftarrow 0$ and $d(v) \leftarrow +\infty$ for all $v \in V \setminus \{s\}$;
Initialize Fibonacci heap H with all vertices of R and key $d(\cdot)$;
while H is not empty **do**

```
    u ← H.ExtractMin()                                              ▷ Step 1
    for all v ∈ Bundle(u) do
        Relax(v, d(u) + dist(u, v));
        for all y ∈ Ball(v) do
            Relax(v, d(y) + dist(y, v));
        for all z_2 ∈ Ball(v) do
            for all z_1 ∈ N(z_2) do
                Relax(v, d(z_1) + w_{z_1,z_2} + dist(z_2, v));
    for all x ∈ Bundle(u) do                                        ▷ Step 2
        for all y ∈ N(x) do
            Relax(y, d(x) + w_{x,y});
        for all z_1 ∈ Ball(y) do
            Relax(z_1, d(x) + w_{x,y} + dist(y, z_1));
Function: Relax(v, D)
if D < d(v) then
    d(v) ← D;
    if v ∈ H then
        H.DecreaseKey(v, D);
    v ∉ R
    Relax(b(v), d(v) + dist(v, b(v)));                              ▷ Step 3
```

§3 Proof of Correctness

We hold two propositions in parallel:

Proposition 3.1 (Distance Correctness)
The ith vertex drawn from R is the ith closest vertex to s, and its distance function is correct when drawn.

Proposition 3.2 (Bundle Correctness)
After Step 1, the distance function is correct for all $v \in \text{Bundle}(u)$ for all $u \in R$ already drawn from the priority queue.

Both for a base case and by way of example, we prove this for the 0th iteration.

Example 3.3 (Base case)
On the 0th iteration, we draw the source node s from the PQ.

To show Proposition 3.1, s is trivially the closest node to s, and its distance function is correct.

To show Proposition 3.2, note that for all $v \in \text{Bundle}(s)$ we already computed $\text{dist}(v, s)$ in the setup Dijkstra, so $d(v)$ is set to the correct value $\text{dist}(v, s)$.

Now we show Proposition 3.1 in generality. See Figure 3 for reference.

Let the current time be t. By induction the $t-1$ closest nodes u_i to s in R have already been drawn. We claim that before step 1, some node u tied for t-th closest to s must satisfy $d(u) = \text{dist}(u)$.

Take any such t-th closest node u, and consider the shortest path from s to u. The set of nodes Bundle(u_i) for $i < t$ have all been processed, so take the last such $x \in$ Bundle(u_i) on the path, and let y be its successor. Then $b(y) = u_*$ is at least as close to y as u is, so $\text{dist}(u_*) \leq \text{dist}(u)$. However u_* hasn't been processed yet, so $\text{dist}(u_*) \geq \text{dist}(u)$ and hence u_* is either equal to u or is also tied for t-th closest to s. We claim $d(u_*) = \text{dist}(u_*)$.

Since u_i was already processed on iteration $i < t$, by the inductive hypothesis of Proposition 3.2 we have $\text{dist}(s, x) = d(x)$. Furthermore, in step 2 of Algorithm 2.6 on u_i we already relaxed $y \in N(x)$ by x, so $d(y) = \text{dist}(y)$ and finally by step 3 we relaxed u_* through y and so $d(u_*) = \text{dist}(u_*)$, and our claim is shown using $u = u_*$.

Thus at the beginning of step 1, at least one of the t-closest nodes satisfies $d(u) = \text{dist}(u)$. So the node u_t drawn from the priority queue must be one of these t-closest nodes, and must already satisfy $d(u_t) = \text{dist}(u_t)$, and the proposition is shown.

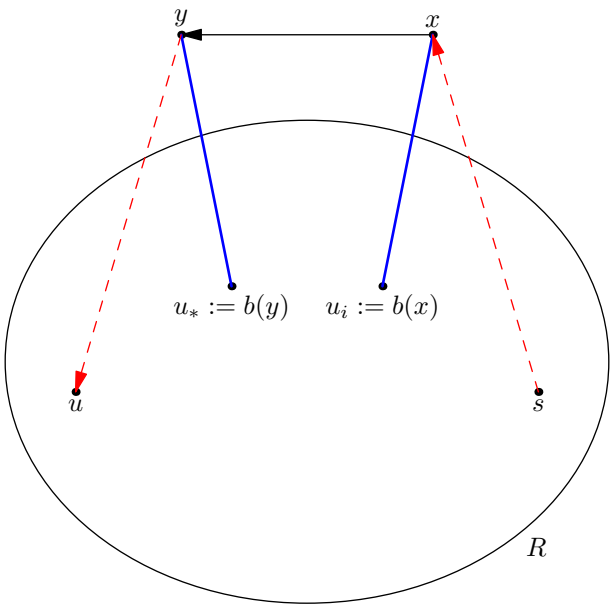

Figure 3: All nodes on the shortest path belong to previously processed bundles or bundles with the same distance to their representative.

Now we show Proposition 3.2 in generality. We denote the t-th node drawn from R, u_t, by just u.

Take any $v \in$ Bundle(u). Once again consider the true shortest path $s \to v$. Like above take the last point x belonging to an already-processed bundle (i.e. $b(x) = u_i$ for some $i < t$), and consider x's successor y. Recall that $u_* := b(y)$ must satisfy $\text{dist}(u_*) \geq \text{dist}(u)$ because u_* has not been processed yet.

Then note that (refer to Figure 4)

$$\begin{aligned}
\text{dist}(s, y) + \text{dist}(y, u_*) + \text{dist}(u, v) &\geq \text{dist}(s, u_*) + \text{dist}(u, v) \\
&\geq \text{dist}(s, u) + \text{dist}(u, v) \\
&\geq \text{dist}(s, v) \\
&= \text{dist}(s, y) + \text{dist}(y, v).
\end{aligned}$$

Thus $\text{dist}(y, v) \leq \text{dist}(y, u_*) + \text{dist}(v, u)$. Hence when we walk from y to v, the first node z_2 that lies outside of Ball(y) must lie in Ball(v). Denote the predecessor of z_2 by z_1, so $z_1 \in$ Ball(y).

Note that in step 2 of Algorithm 2.6 while processing u_i, we relaxed on the ball of every neighbor of every node in Bundle($b(x)$), so walking from $s \to x \to y \to z_1$ we have $d(z_1) = \text{dist}(s, z_1)$.

Similarly, note that during the third bullet point of step 1 of Algorithm 2.6 on Bundle(u), we relaxed on v through every element of Ball(v) and its neighbor. Hence, we relax $v \leftarrow z_2 \leftarrow z_1$. Therefore, by the end of step 1, $d(v)$ is correct.

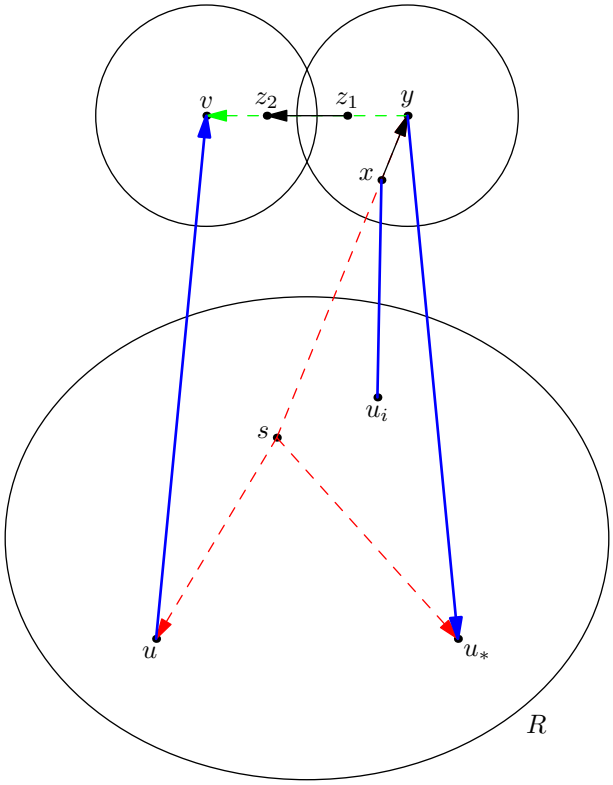

Figure 4: If x lies on the shortest path from $s \to y$ and if u_i has been processed but u_* has not, then $\text{dist}(s, u_*) \geq \text{dist}(s, u_i)$. Then $\text{dist}(x, y) \leq \text{dist}(x, u_*) + \text{dist}(y, u_i)$. Then we get neighbors z_1, z_2 satisfying $z_1 \in \text{Ball}(x), z_2 \in \text{Ball}(y)$.

Thus we have proven both Proposition 3.1 and Proposition 3.2, so eventually $d(v) = \text{dist}(v)$ for all $v \in G$, and algorithm correctness has been shown.

§4 Time complexity analysis

To ensure the time complexity of *Bundle Dijkstra*, we need to first transform the graph into a graph with small degrees. For a given graph G, we can construct G' in the following way:

- For each vertex v and $w \in N(v)$, construct the vertex x_{vw}. Add 0-weight edges between these $|N(v)|$ vertices so that they form a cycle.
- For every edge $(u, v) \in G$, add an edge between x_{uv} and x_{vu} with weight w_{uv}.

We can see that $\text{dist}_{G'}(x_{uu'}, x_{vv'}) = \text{dist}_G(u, v)$ for all $u, v \in G$ and $u' \in N(u), v' \in N(v)$. Moreover, in this construction we have $O(m)$ vertices and each of them has degree at most 3.

§4.1 Bundle Construction

Without loss of generality, we assume that Dijkstra's algorithm breaks ties deterministically. Let S_v be the set of vertices we extracted when we run Dijkstra on v. When we extract a vertex u from the heap, the probability that $u \in R$ is $\frac{|R|}{n} = \frac{1}{k}$. As our Dijkstra terminates when we find such u, we have $\mathbb{E}[|S_v|] = k$. Moreover, by linearity of expectation, we have:

$$\mathbb{E}[|S_v|^2] = \frac{1}{k} \cdot 1^2 + (1 - \frac{1}{k})\mathbb{E}[(|S_v| + 1)^2] = \frac{1}{k} + (1 - \frac{1}{k})(1^2 + 2\mathbb{E}[|S_v|] + \mathbb{E}[|S_v|^2])$$

Simplify and we get $\mathbb{E}[|S_v|^2] = 2k^2 - k = O(k^2)$. Let $f(x) = \sqrt{x}\log x$, we have $f''(x) = -\frac{\log(x)}{4x^{3/2}} \le 0$ for $x \ge 1$. Then by Jensen's inequality,

$$\mathbb{E}[|S_v| \log |S_v|] \le f(\mathbb{E}[|S_v|^2]) = O(k \log k)$$

Thus, the total runtime of all Dijkstra is

$$O(\sum_{v \in V \setminus R} \mathbb{E}[|S_v| \log |S_v|]) = O(mk \log k)$$

in expectation. The same bound applies to $\text{Ball}(v)$ since it is a subset of S_v.

§4.2 Bundle Dijkstra

The total runtime of extract-min of the Fibonacci heap is $O(|R| \log n)$. Every vertex $v \notin R$ only appears once in step 1 as v, in step 2 as x when $v \in \text{Bundle}(u)$. Since every vertex in the graph has a constant degree, it appears a constant number of times as y, a neighbor of x, in step 2. Similarly, one can argue that we have $O(|\text{Ball}(v)|)$ (z_1, z_2) pairs and $O(|\text{Ball}(v)|)$ z_1 in step 2. The overall runtime of the algorithm is thus

$$O(|R| \log n + \sum_{v \in V \setminus R} |\text{Ball}(v)|) = O(\frac{m}{k} \log k + mk \log k)$$

in expectation. Take $k = \sqrt{\frac{\log n}{\log \log n}}$, and we get:

$$O(\frac{m}{k} \log k + mk \log k) = O(m\sqrt{\frac{\log n}{\log \log n}} \log \log n) = O(m\sqrt{\log n \log \log n})$$

§5 Improved Bundle Construction

In Section 4 we showed that the time complexity for Bundle Construction is $O(\sum_{v \notin R} |S_v| \log |S_v|)$ and Bundle Dijkstra is $O(R \log n + \sum_{v \notin R} |\text{Ball}(v)|)$. In this section, we will improve Bundle Construction so that it runs in the desired time complexity $O(m\sqrt{\log n \cdot \log \log n})$ with high probability while maintaining the properties as desired.

Algorithm 5.1 (Improved Bundle Construction) — The algorithm is as follows:

1. Sample vertex $v \in V \setminus \{s\}$ into R_1 independently with probability $\frac{1}{k}$.
2. For each $v \in V \setminus R_1$, run Dijkstra from v until we meet a vertex in R_1; or have already popped $k \log k$ vertices.

3. In the former case we do not change; in the latter case we add v to R_2.
4. Set $R = R_1 \cup R_2$.
5. Calculate $\text{Ball}(v)$ and $b(v)$ for all $v \in V \setminus R$ and get $\text{Bundle}(u)$.

We will prove the following proposition:

Proposition 5.2

With probability $O(1 - e^{-\Omega(n^{1-o(1)})})$, these properties hold:

1. $|R| = O(m/k)$
2. $\sum_{v \in V \setminus R} |\text{Ball}(v)| = O(mk)$
3. The running time of the algorithm Algorithm 5.1 is $O(mk \log k)$.

First, we need to use some theorems in probablistic theory without proofs (since we do not care about mathematical details):

Theorem 5.3

Suppose $X_1, X_2, \ldots, X_n$ are n random independent variables taking values in $\{0, 1\}$. Let $X = \sum X_i, \mu = \mathbb{E}[X]$, then for any $\delta > 0$:

$$\Pr(X \geq ((1+\delta)\mu)) \leq \exp(-\delta^2\mu/(2+\delta))$$

Intuitively, this tells us that the sum of n random independent variables is likely to be around its expected value.

Lemma 5.4

Suppose a set of random variables $\{Z_v\}_{v \in S}$ satisfy that for each $v \in S$, $\mathbb{E}[Z_v] = \mu$, $Z_v \in [0, b]$ with probability 1, and each Z_v is corresponded to a fixed deterministic set W_v such that if all W_v are disjoint, Z_v are independent. Suppose W_v intersects with at most T other W_u. Then, with probability

$$1 - 8Tb\mu^{-1} \cdot e^{-\mu^3|S|/(8b^3T)}$$

it holds that $\sum_{v \in S} Z_v = O(|S|\mu)$.

Proof. Notice that the runtime of the algorithm is based on the first two propositions, so it suffices to prove the first two propositions.

First, by Chernoff bound, we have $|R_1| = \Theta(m/k)$ with probability $1 - O(e^{-m/k})$ since each v is inserted in R_1 independently.

For each $v \in V \setminus R_1$, define $X_v = \mathbb{I}(v \in R_2)$ and Y_v the number of vertices in the Dijkstra step.

$$\mathbb{E}[X_v] = (1 - \frac{1}{k})^{k \log k} = \Theta(\frac{1}{k})$$

Since originally $\mathbb{E}[Y_v] = k$, the truncation will only make the expected value smaller. Thus $\mathbb{E}[Y_v] = \Theta(k)$.

Let S_v be the first $k \log k$ vertices extracted in the Dijkstra algorithm if it did not truncate. Notice that the set S_v depends only on the structure of G so there is no randomness. The values of X_v and Y_v depends on whether vertices in S_v are inserted to R_1. If each S_v are disjoint, then all X_v and Y_v become independent.

For each vertex $w \in S_v$, because w is found in $k \log k$ steps of Dijkstra's algorithm, the number of edges between w and v is no more than $k \log k$. By constant degree property, there are at most

$$3 \cdot (1 + 2 + \cdots + 2^{k \log k - 1}) \leq 3 \cdot 2^{k \log k}$$

u such that $w \in S_u$. Thus, there are at most

$$(k \log k) 3 \cdot 2^{k \log k} = O(n^{o(1)})$$

different $u \in V \setminus R_1$ such that $S_u \cap S_v \neq \emptyset$.

Now, we apply Lemma 5.4 for X_v and Y_v to get the desired result. With $T = O(n^{o(1)})$, we can verify that $8Tb\mu^{-1} = O(n^{o(1)})$ and $8b^3T/\mu^3 = O(n^{o(1)})$. This finishes the proof. □

§5.1 Pseudocode

Algorithm 5.5 — Setup for the algorithm.

Input: A graph $G = (V, E, w)$, source vertex $s \in V$, and parameter k
Output: Bundles and sets R, $Ball(v)$, $Bundle(u)$, $b(v)$
Initialize $R_1 = \{s\}$ and insert each $v \in V \setminus \{s\}$ independently to R with probability $\frac{1}{k}$.
for all $v \in V \setminus R_1$ **do**
    Initialize Fibonacci heap $H^{(v)}$ with vertex v and its key $d^{(v)}(v) \leftarrow 0$
    Initialize an ordered list $V^{(v)}_{\text{extract}} \leftarrow \emptyset$
    while $H^{(v)}$ is not empty **do**
        $u \leftarrow H^{(v)}$.ExtractMin()
        Append u to $V^{(v)}_{\text{extract}}$
        if $u \in R$ **then**
            break the while-loop
        else if $|V^{(v)}_{\text{extract}}| > k \log k$ **then**
            set R_2 to $R_2 \cup \{v\}$ and **break the while-loop**
        else
            for all $x \in N(u)$ **do**
                if $x \notin H^{(v)}$ and $x \notin V^{(v)}_{\text{extract}}$ **then**
                    $H^{(v)}$.Insert$(x, d^{(v)}(u) + w_{ux})$
                else if $d^{(v)}(x) > d^{(v)}(u) + w_{ux}$ **then**
                    $H^{(v)}$.DecreaseKey$(x, d^{(v)}(u) + w_{ux})$
$R \leftarrow R_1 \cap R_2$
for all $v \in V \setminus R$ **do**
    $b(v) \leftarrow$ the first vertex in the ordered list $V^{(v)}_{\text{extract}}$ that lies in R
Compute Bundle(u) for all $u \in R$, Ball(v) for all $v \in V \setminus R$, and record dist(v, u) for all $u \in$ Ball$(v) \cup \{b(v)\}$

References

[1] Ran Duan, Jiayi Mao, Xinkai Shu, and Longhui Yin. A Randomized Algorithm for Single-Source Shortest Path on Undirected Real-Weighted Graphs . In *2023*

IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS), pages 484–492, Los Alamitos, CA, USA, November 2023. IEEE Computer Society. URL https://doi.ieeecomputersociety.org/10.1109/FOCS57990.2023.00035.